

names, leading to seemingly paradoxical code like this:

```
Python 2.7.14 (v2.7.14:84471935ed, Sep 16 2017, 20:25:58) [MSC v.1500 64 bit
(AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> True is False
False
>>> True = False
>>> True is False
True
```

Fortunately, this sort of confusing code isn't possible in Python 3, which will raise a syntax error if you try to use the keywords `True` or `False` as variable names.

Chaining Multiple Kinds of Operators

Chaining different kinds of operators in the same expression can produce unexpected bugs. For example, this (admittedly unrealistic) example uses the `==` and `in` operators in a single expression:

```
>>> False == False in [False]
True
```

This `True` result is surprising, because you would expect it to evaluate as either:

- `(False == False) in [False]`, which is `False`.
- `False == (False in [False])`, which is also `False`.

But `False == False in [False]` isn't equivalent to either of these expressions. Rather, it's equivalent to `(False == False) and (False in [False])`, just as `42 < spam < 99` is equivalent to `(42 < spam) and (spam < 99)`. This expression evaluates according to the following diagram: